

Arquitectura completa y familias de modelos.

Ya saben cómo un transformer presta atención. Hoy armamos el bloque completo y decidimos cuál familia — encoder, decoder o encoder-decoder — le sirve a su proyecto.

[BLOQUE COMPLETO](#)[→ 3 FAMILIAS](#)[→ ELEGIR MODELO](#)

La sesión de hoy.



Sus dudas de la sesión pasada



Recap y pregunta del paper



El bloque transformer completo + Lab A



Las tres familias + Lab B



Objetivos de pre-entrenamiento + Lab C



Escalado y leyes empíricas



Para pensar: qué modelo elige tu equipo

BLOQUE 01

Sus dudas

Respuestas del formulario de salida

¿Por qué $\text{dim} = -1$?

El eje que nombras en dim es el eje que RECORRES. Y lo que recorres es lo que suma 1.

$\text{dim} = -1$

recorre las COLUMNAS

→ cada FILA suma 1

Cada palabra reparte el 100% de SU atención entre las demás. Es lo que queremos.

$\text{dim} = -2$

recorre las FILAS

→ cada COLUMNA suma 1

Cuánta atención RECIBE cada palabra. Cuenta válida, pregunta distinta. Corre sin error y da otra cosa.

La pregunta que lo resuelve siempre: ¿qué quiero que sume 1? Si la respuesta es «cada fila», recorro las columnas.

¿Sobre qué eje va el softmax?

La matriz de atención: cada **fila** es una palabra, cada **columna** es una palabra a la que mira. El **dim** decide en qué dirección se reparte el 100%.

softmax(dim=-1)

softmax(dim=-2)

	el	gato	duerme	
el →	0.23	0.57	0.20	1.0 fila «el» suma 1
gato →	0.05	0.90	0.05	1.0 fila «gato» suma 1
duerme →	0.10	0.15	0.75	1.0 fila «duerme» suma 1

Recorre las **columnas** → cada **fila** suma 1.
Cada palabra reparte el 100% de su atención. ☒ lo que queremos

► recorrer de nuevo

En la atención queremos **dim=-1**: cada palabra reparte el 100% de SU atención. **dim=-2** corre sin error pero calcula otra cosa — por eso es el bug que no explota.

Y los números negativos

No son magia: cuentan los ejes desde el final en vez de desde el principio.

(1 , 3 , 2)

frases palabras números

desde adelante:	0	1	2
desde atrás:	-3	-2	-1

	filas es el eje...	columnas es el eje...
tensor (3, 2) sin batch	0	1
tensor (1, 3, 2) con batch	1	2
	cambia ✗	cambia ✗

	filas	columnas
tensor (3, 2)	-2	-1
tensor (1, 3, 2)	-2	-1
	igual ✓	igual ✓

Contando desde atrás, las filas SIEMPRE son -2 y las columnas SIEMPRE son -1. Como decir «el último día del mes» en vez de «el 31».

¿Qué es un "eje"? ¿Y por qué negativos?

Un tensor es una caja de números. Cada forma de recorrerlo es un **eje**. Avanza paso a paso.



el	0.2	0.5	0.1	0.7
gato	0.9	0.1	0.3	0.4
duerme	0.0	0.8	0.6	0.2

3 palabras × 4 números por palabra

1 • Una frase = una tabla

Empecemos con lo simple. Esta tabla es **una frase**: 3 palabras (las **filas**) y 4 números que describen cada palabra (las **columnas**). En PyTorch esto es un tensor de forma **(3, 4)**.

← anterior

siguiente →

una frase

el

gato

duerme

1

2

3

4

5

6

7

8

una frase

RoPE, en una diapositiva

Lo que importa no es en qué posición está una palabra. Es a qué distancia está de otra.

- **El problema**

«no» justo antes de «vigente» cambia todo el sentido.
Que sea la palabra 340 o la 890 no le importa a nadie.

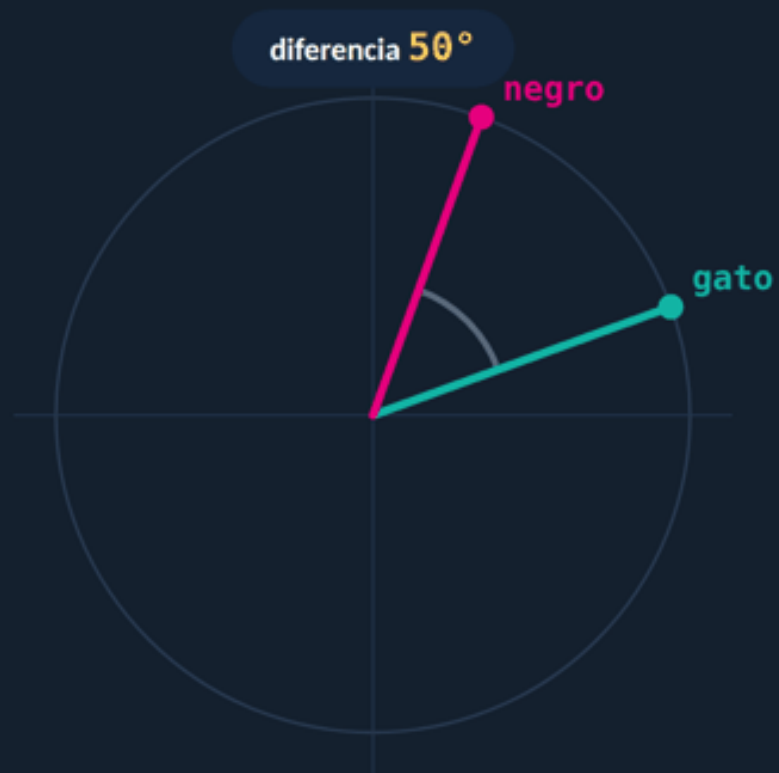
- **La idea**

En vez de SUMAR una señal al vector, lo ROTA un ángulo proporcional a su posición.

- **El truco**

Al comparar dos palabras con producto punto, los ángulos se restan.
Queda solo la DIFERENCIA de posiciones. Lo relativo sale gratis.

dos palabras · posición 0



- 1
- 2
- 3
- 4
- 5
- 6

Dos palabras son dos **flechas**. Entre ellas hay un ángulo.

LLaMA · Qwen · Mistral usan RoPE. La rotación es en muchas dimensiones; aquí se ve en un plano.

«¿Mi agente en español o en inglés?»

Es una decisión de diseño de su sistema.

Todo en español

✓ El usuario escribe y lee en español. Sencillo, coherente, sin traducción.

✗ Paga el sobrecosto de tokens en cada llamada. Menos modelos disponibles.

Traducir por dentro

✓ Entra español, se traduce a inglés, el modelo trabaja barato, se traduce de vuelta.

✗ Dos llamadas más de latencia. La traducción introduce errores propios.

Modelo multilingüe

✓ Un solo modelo que ya tokeniza bien el español. Sin traducción intermedia.

✗ Suelen ser más grandes para la misma calidad en una sola lengua.

No hay respuesta correcta: hay una decisión que ustedes tienen que justificar en M1. Esto es exactamente lo que evalúa el criterio de solidez técnica.

EMPECEMOS

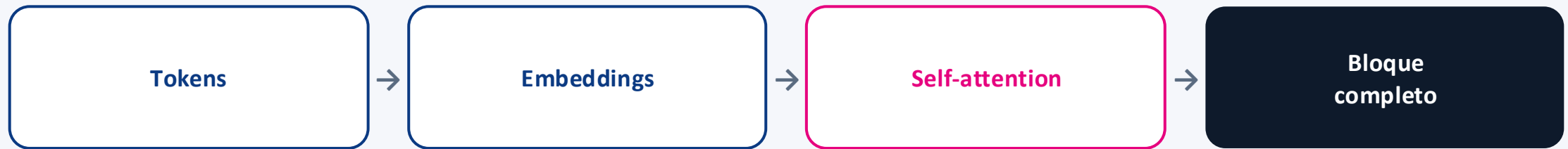
Recap:

En pocas palabras

What's attention?

What does the paper say?

Lo que ya saben: attention.



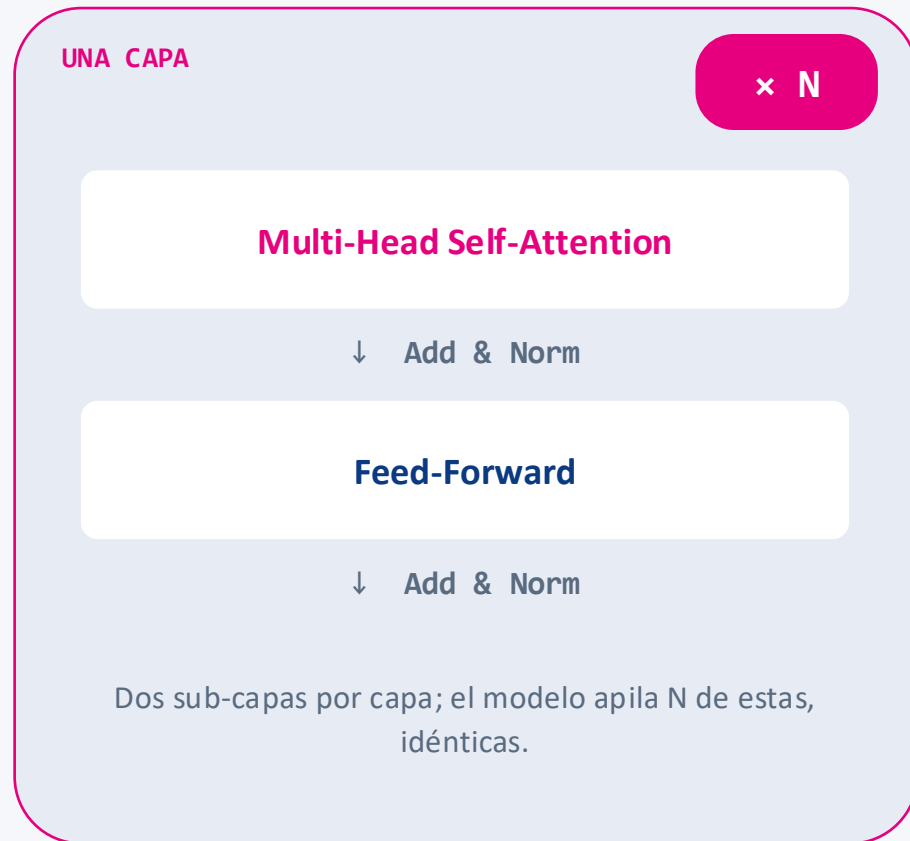
Cada token genera tres vistas: **Q**, **K**, **V**.

Compara su **Q** con las **K** de todos, escala por $\sqrt{d}$, softmax, y mezcla los **V**. Resultado: cada token se **reescribe** mirando a los demás.

$$\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}} \right) \cdot V$$

La pieza que construyeron
en S02, desde cero.

El bloque, en palabras del paper.



La regla exacta del paper:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

- Cada sub-capa se envuelve en una **conexión residual** (ese $x +$) y luego layer norm.
- El residual deja pasar la entrada **intacta** y le suma lo que aporta la sub-capa.
- En el original: **$N = 6$ capas, $d = 512$** . Los modelos de hoy escalan estos dos números.

Ya vieron qué pone el paper.
La pregunta es **por qué**.

¿Por qué envolver cada sub-capas en **residual + layer norm**? ¿Qué se rompería sin eso?

💡 **PISTA: VANISHING GRADIENT**

REDES PROFUNDAS DE LOS PRERREQUISITOS

No necesitan la respuesta exacta ahora: la van a MEDIR ustedes mismos en el Lab A, en un minuto.

El bloque completo.

Multi-head attention no viene sola: va envuelta en residual, norm y una FFN.

Qué hay dentro de una capa.

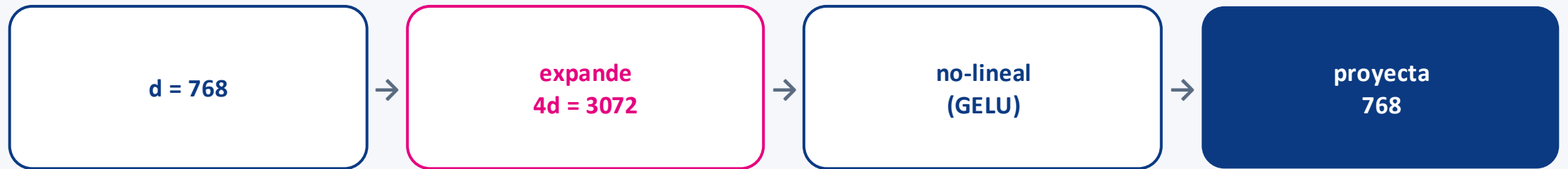


Residual: la entrada se **suma** a la salida de cada sub-bloque. Un atajo para el gradiente.

FFN: dos capas lineales con no-linealidad. Procesa cada token por separado. Aquí viven la mayoría de los parámetros.

Norm: estabiliza la escala en cada capa.

La FFN carga con los parámetros.



La FFN expande a **4x** por dentro ($d \rightarrow 4d$) y termina cargando **dos tercios de los pesos** del bloque — más que la atención. Lo cuentan en el Lab A.

El atajo que deja entrenar en profundidad.

Sin conexión residual, el gradiente tiene que atravesar decenas de capas multiplicándose en cada una: se desvanece antes de llegar abajo. El residual le da una autopista directa ($x + f(x)$) que el gradiente recorre casi intacto.

Es el mismo **vanishing gradient** que vieron en Redes Neuronales — y la misma solución que hizo posible ResNet. En un momento lo rompemos en el notebook para verlo medido.

Al notebook — Lab A

El bloque, armado y roto

Construyen un bloque transformer en PyTorch (attention + residual + norm + FFN). Completan la conexión residual, apilan decenas de capas y miden la norma del gradiente en la primera capa — con y sin residual.

```
grad_norm_con_residual vs grad_norm_sin_residual → S03_Lab_El_bloque_y_las_familias.ipynb
```

Qué acaban de medir.

- **Con residual** el gradiente llega a la capa 1 con una norma sana; la red profunda entrena.
- **Sin residual** la señal se debilita mucho: la red profunda casi no aprende en las capas bajas.
- **La FFN** confirmaron contando parámetros que pesa más que la atención dentro del bloque.

Tres familias.

Mismo bloque, distinto cableado: quién puede mirar a quién lo cambia todo.

Tres formas de apilar el mismo bloque.

ENCODER-ONLY



Mira toda la frase

Atención bidireccional. Bueno para clasificar y extraer.

BERT • RoBERTa • DeBERTa

DECODER-ONLY



Solo mira hacia atrás

Atención causal (máscara triangular). Bueno para generar.

GPT • LLaMA • Mistral • Qwen

ENCODER-DECODER

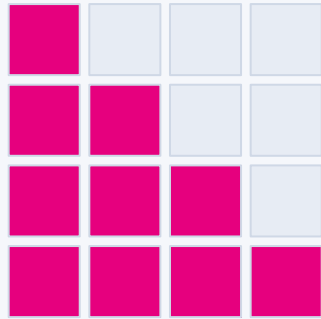


Lee, luego escribe

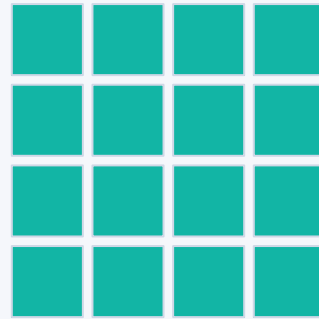
Un encoder comprime, un decoder genera. Bueno para seq2seq.

T5 • BART • FlanT5

La máscara decide la familia.



Causal (decoder)



Bidireccional
(encoder)

Clasificar sentimiento

Encoder

Extraer entidades / QA

Encoder

Generar / chatear

Decoder

Traducir / resumir

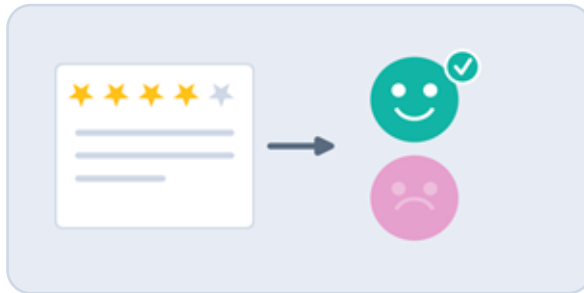
Enc-Dec

Y en la práctica, ¿qué hace cada una con un caso real? Veámoslo →

¿Cómo se ve cada familia en la práctica?

ENCODER

Clasificar / rellenar



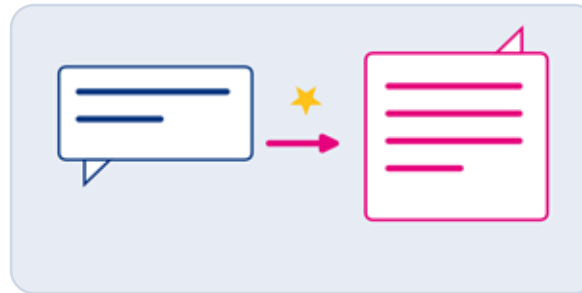
"La entrega llegó fría."

→ **NEGATIVA**

Uso real: moderar reseñas, filtrar spam, extraer datos de un texto.

DECODER

Generar / conversar



"Disculpa al cliente:"

→ **"Lamentamos mucho..."**

Uso real: chatbots, asistentes, redacción automática.

ENCODER-DECODER

Transformar texto → texto



"Resume esto: ..."

→ **resumen de 1 frase**

Uso real: traducir, resumir, reescribir en otro tono.

Al notebook — Lab B

Tres familias, un mismo problema

Le dan la MISMA frase a BERT, a Qwen y a T5. Ven qué hace cada uno con ella, comparan tamaño y salida en una tabla, y prueban con texto real de su dominio para decidir qué familia les sirve.

```
fill_mask(BERT) • generate(Qwen) • seq2seq(FlanT5) → Lab B
```

Una frase entra, salen tres cosas distintas:

FAMILIA	LE DIMOS	DEVOLVIÓ	QUÉ NOS DICE
 Encoder BERT	"...Francia es [MASK]."	→ "París"	Entiende el contexto; no escribe libre.
 Decoder Qwen	"...Francia es"	→ continúa la frase	Genera fluido; no fue hecho para clasificar.
 Enc-dec T5	"translate: capital..."	→ "...capitale..."	Sigue la instrucción; salida estructurada.

El bloque es el mismo. Lo que cambia – quién mira a quién – decide qué sabe hacer cada modelo.

¿De dónde sale eso?

Cada familia es buena en algo porque fue entrenada con un objetivo distinto.

Cada familia entrena resolviendo un juego distinto.

MLM

Masked LM · Encoder

El

gato

[MASK]

en

la

alfombra

Tapa un token y lo adivina viendo AMBOS lados → aprende a entender contexto.

CLM

Causal LM · Decoder

El

gato

duerme

en

la

[?]

Predice el siguiente token usando SOLO lo anterior → aprende a generar.

Span

Span corruption ·
Enc-dec

El

gato

<extra_id_0>

la

alfombra

Borra un tramo entero y lo reconstruye → aprende a transformar texto en texto.

Al notebook — Lab C

El objetivo, en vivo

Los mismos tres modelos, ahora resolviendo el objetivo con el que fueron entrenados: BERT rellena un [MASK], Qwen muestra la distribución del siguiente token, T5 completa un <extra_id_0>. El objetivo, hecho código.

```
top_k(masked) • next_token_probs • fill(<extra_id_0>) → Lab C
```

Elegir el modelo base no es elegir el más grande.

El objetivo de pre-entrenamiento ya dejó al modelo bueno para cierto tipo de tarea. En S04 van a fine-tuneear un modelo base para su proyecto: conviene empezar de la familia cuyo objetivo se parece a lo que necesitan.

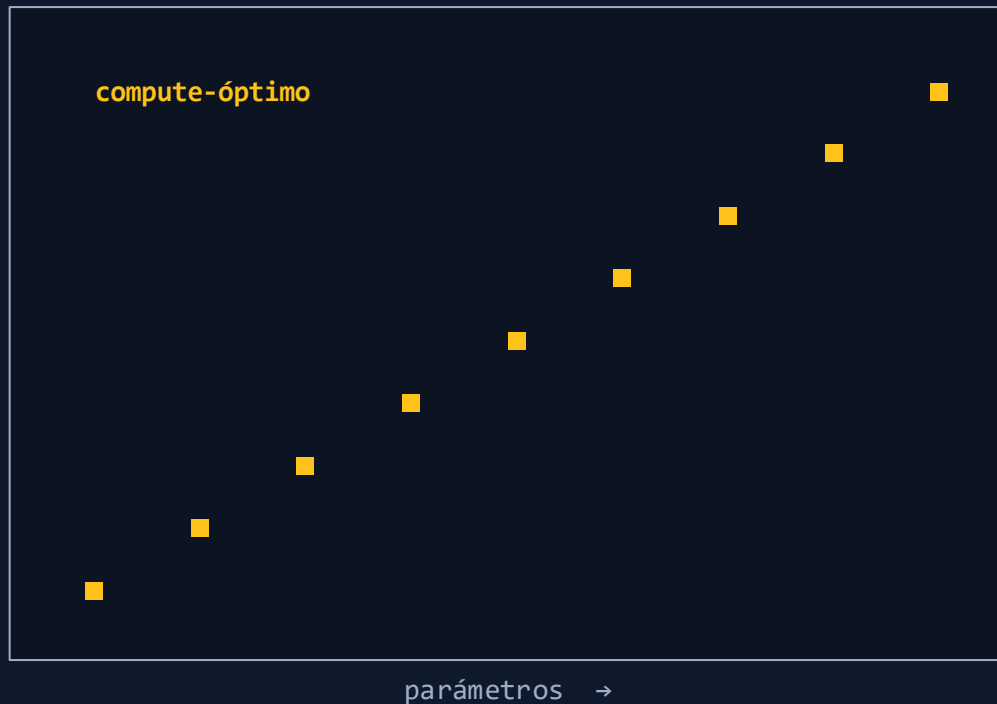
Regla práctica: clasifican/extraen → arranquen de un encoder (DistilBERT). Generan/conversan → un decoder pequeño (TinyLlama, Qwen-0.5B). Transforman texto a texto → FlanT5-small.

¿Y el tamaño?

Más grande no siempre es mejor. Las leyes empíricas dicen qué compensa.

Parámetros vs. datos de entrenamiento.

↑ tokens



La conclusión práctica:

Para fine-tunear no necesitan el modelo más grande. Necesitan el modelo **correcto**, entrenado con suficientes datos.

Un modelo pequeño bien elegido gana en Colab, en costo y en velocidad.

Mixture of Experts, en una idea.

En vez de una FFN gigante, muchas FFN “expertas” y un router que enciende solo una o dos por token. El modelo tiene muchísimos parámetros, pero cada token usa una fracción.

Mixtral 8x7B

DBRX

router

solo 2 de 8 expertos activos

Para ustedes es cultura general por ahora: no van a entrenar un MoE, pero verán el nombre en los modelos del Hub y conviene saber qué promete.

Para pensar.

La decisión que se llevan de hoy, y lo que trae S04.

¿Qué **familia** y qué **modelo base** candidato usará su proyecto en M1?

Lo declaran hoy: familia elegida + un modelo del Hub para arrancar.

No es definitivo — pero en S04 llegan con un candidato para fine-tunear, no con la página en blanco.

Lo que vino después de 2017.

GQA

Grouped-Query Attention

Comparte keys/values entre cabezas.
Menos memoria en inferencia. Llama 3.

SWA

Sliding-Window Attention

Cada token solo mira una ventana local. Contextos largos baratos. Mistral.

SSM

Mamba / espacios de estado

Alternativa a la atención, coste lineal en la longitud. Aún experimental.

Mensaje: el transformer es la mejor herramienta por ahora, no la definitiva.

Tarea para S04

Lleguen listos para entrenar.

1

Confirмен su familia y modelo base candidato del Hub (huggingface.co/models).

2

Consigan un poco de texto real de su dominio: en S04 lo usan para el baseline de M1.

3

Recuerden: S04 entrega M1 (10%) — el primer fine-tuning del proyecto.

**Ya saben qué modelo
le sirve a **su proyecto**.**

La próxima semana: Hugging Face, fine-tuning y LoRA — y entregan M1.